

FlatScan

Zero-Dependency Static Malware Analysis Engine

Fast triage, precise IOCs, and executive
reporting-without executing the sample.

Requires Technical Evidence

Hashes, Strings, IOCs,
Entropy, YARA

JSON

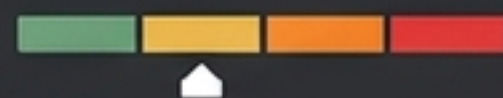
CLI

Interactive HTML

The Pure-Go Engine

One static pipeline serving
both audiences safely.

Requires Risk Context



PDF Reports

MITRE Matrices

Risk Scores



Zero Dependencies

Pure Go standard library. No go.mod bloat. Cross-platform portable across Linux, macOS, and Windows.



Static Only

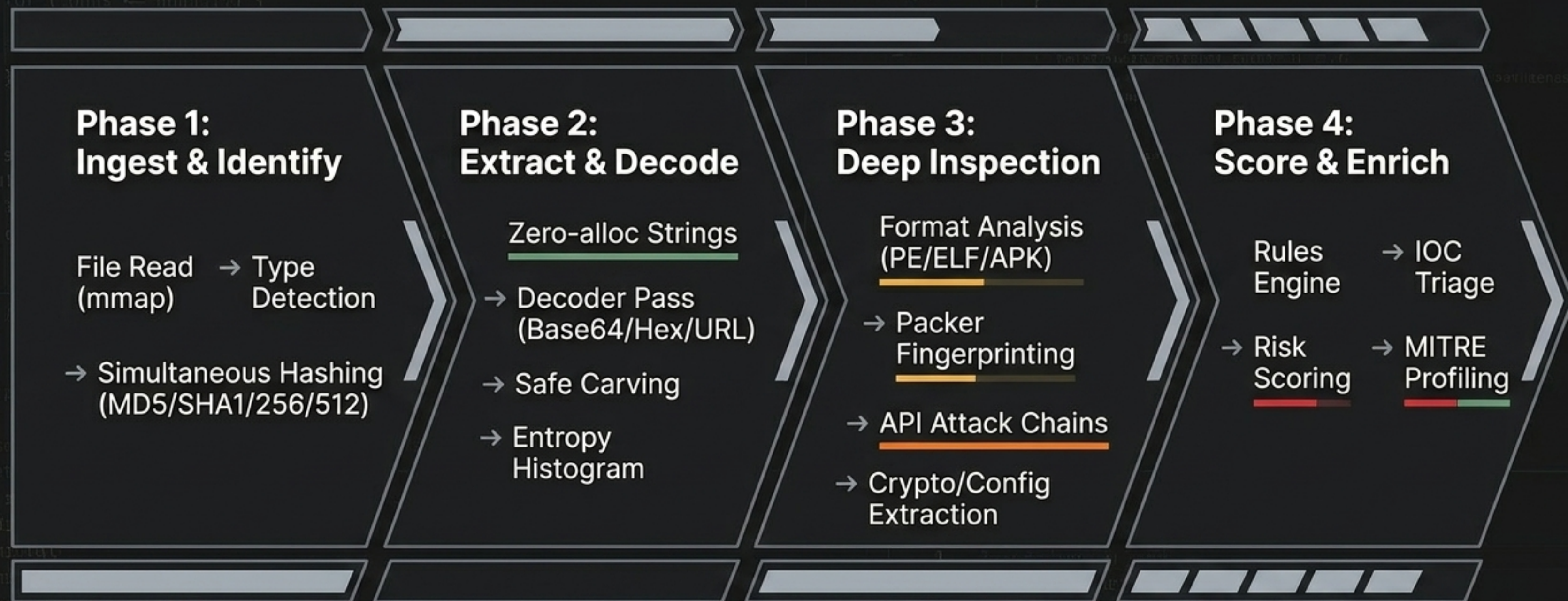
Reads bytes and metadata only. Never executes the sample. Zero risk of sandbox escape.



High Performance

Thread-safe parallel execution, zero-alloc string extraction, and syscall.Mmap memory-mapped I/O for files >100MB.

Threat Analysis Pipeline: From Bytes to Context



Performance Optimizations

<u>Component</u>	<u>Before</u>	<u>After</u>	<u>Impact</u>
Corpus Build	5 independent builds (~240MB)	1 shared build (~48MB)	5x Memory Reduction
String Extraction	Per-string heap allocation	Direct byte-slice indexing	Zero Allocations
Entropy Calculation	O(window) per step	Incremental sliding window O(step)	2x Faster
Analysis Pipeline	Sequential stages	4 parallel goroutines via parallelRun()	~40% Faster on Multi-Core

The v0.5.0 Evolution

Behavioral API Chains

7 multi-stage attack sequences detected statically (e.g., Process Hollowing, Named Pipe C2 + Injection).

Packer Fingerprinting

Section-name and overlay detection for UPX, Themida, VMProtect, MPRESS, and ASPack.

New Threat Families

Statically maps Wiper strings (vssadmin delete) and Cryptominer libraries (stratum+tcp, xmrig) to MITRE tactics.

Richer IOCs

Native extraction of Ethereum/Monero/Bitcoin wallets, Mutex names (Global\...), and Named Pipes (\\.pipe\...).

Scoring Logic & Semantic Exit Codes

```
func score(ctx) {  
    score = exit_  
    return exit_  
}
```

Score breakdown: [Credential Access:44, Evasion:31, Packing:24]

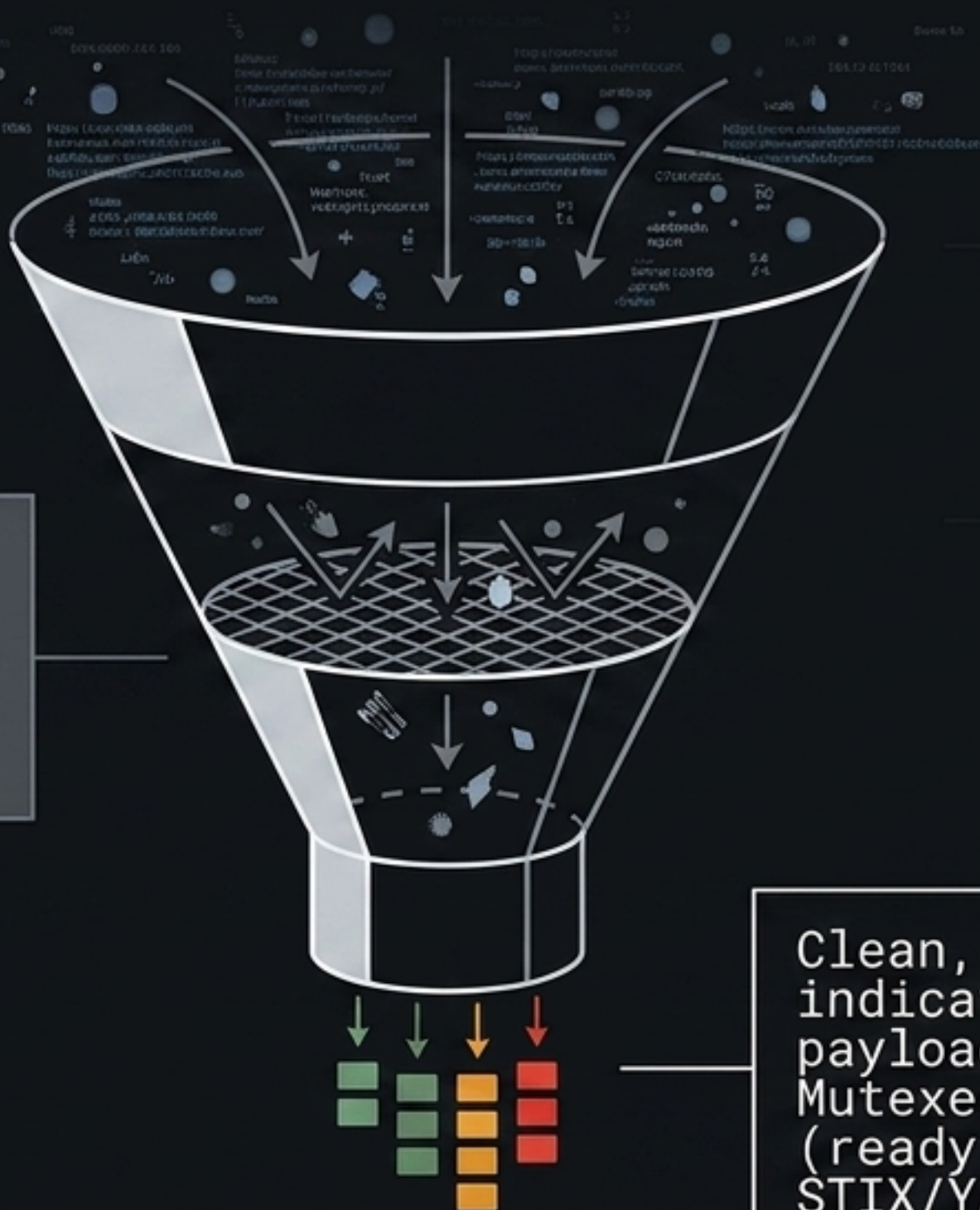
```
func score(ctx) {  
    return exit_code  
}
```

0-29	30-54	55-79	80-100
Low Suspicion / Clean	Suspicious Correlate with telemetry	High Suspicion Treat as high risk	Likely Malicious Prioritize containment
Exit Code 0	Exit Code 10	Exit Code 10	Exit Code 20

The IOC Extraction Funnel

Millions of raw strings, URLs, IPs, and file hashes

Suppression layer actively filtering benign PKI, Microsoft XML schemas, W3C namespaces, OIDs, and loopback addresses.



Clean, ready-to-use indicators: Promoted embedded payload hashes, Crypto Wallets, Mutexes, and C2 endpoints (ready for STIX/YARA/Sigma).

Operational Modes

Direct / Interactive

Wizard-guided triage and deep dives.

```
./flatscan --interactive
```

Batch Mode

Parallel directory-wide triage with worker pools (NumCPU) and JSON summaries.

```
./flatscan --dir ./samples --batch-json out.json
```

Watch Mode

Continuous folder monitoring, silently skipping clean files and alerting on threats ≥ 55 .

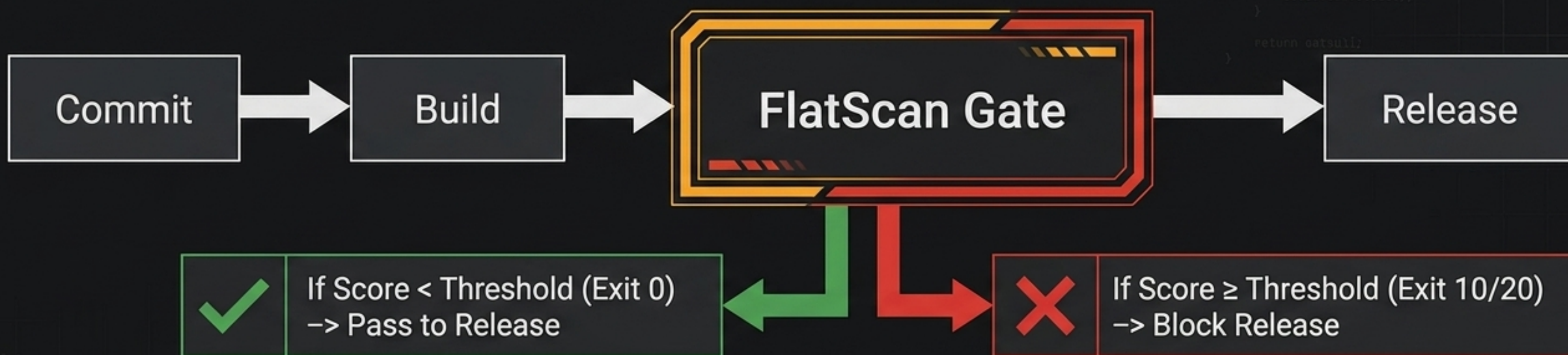
```
./flatscan --dir ./inbox --watch --watch-alert-only
```

CI/CD Mode

Automated pipeline gates with semantic exit codes.

```
./flatscan --ci --ci-threshold 55
```

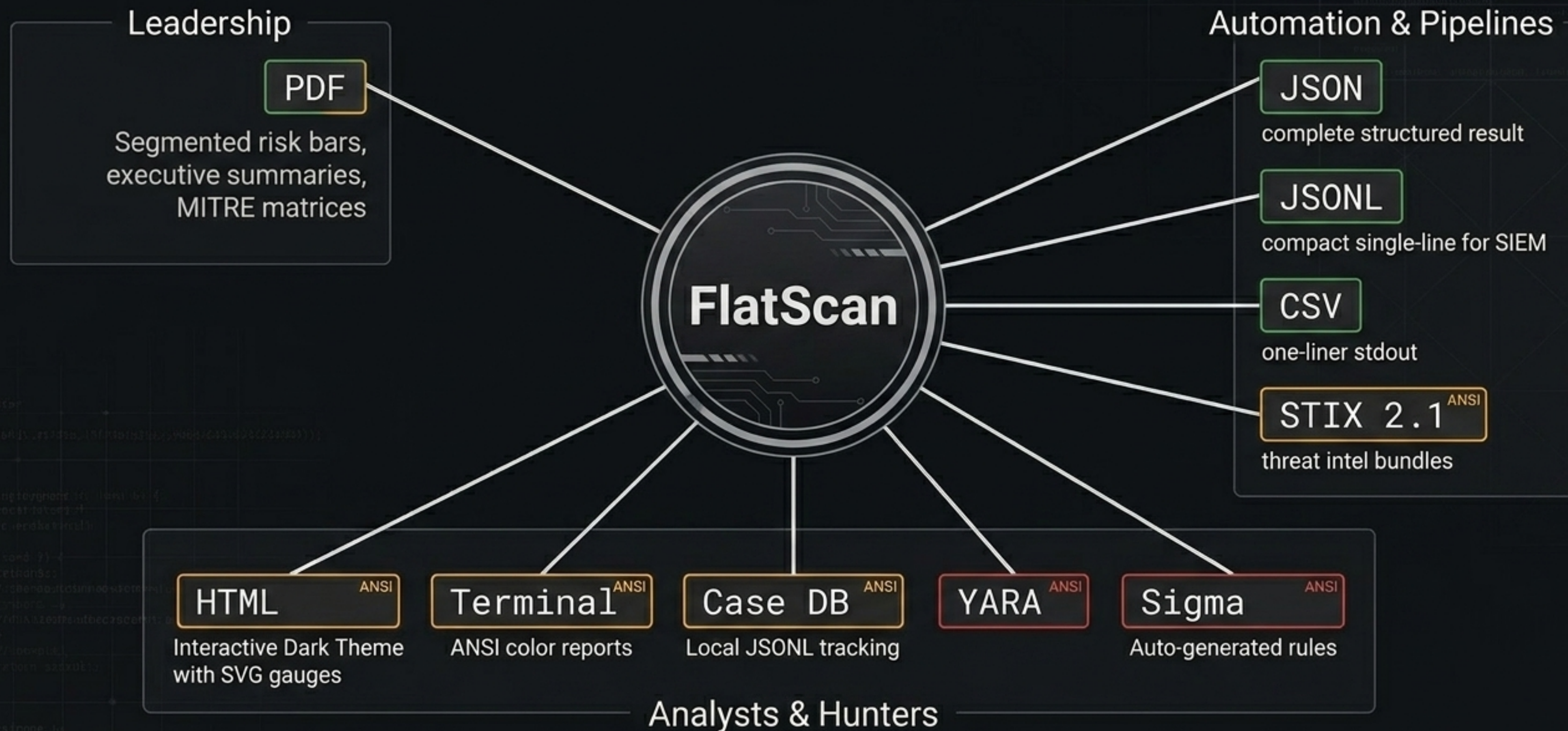

DevSecOps Integration: CI/CD Pipeline Gate



```
> ./flatscan -f build.exe --ci --ci-threshold 30
FLATSCAN: MALICIOUS score=82 file=build.exe findings=13 sha256=...
```

Completely suppresses UI/progress bars. Utilizes new `--output-format csv|jsonl` for SIEM streaming and jq pipelines.

Output Format Ecosystem



Security & Handling Guidelines



FlatScan is static. It does NOT execute samples.

Technical Limitations

Cannot decode or reverse cryptographic hashes.

May miss dynamically staged, heavily packed, or environment-gated execution.

Safe Handling Mandates

Always work inside isolated malware-analysis VMs.

Keep raw samples in password-protected archives.

Treat generated reports (JSON/HTML/PDF) as sensitive incident artifacts containing extracted malicious strings.

Quick Start



1. Build from source (Zero dependencies)

```
go build -o flatscan .
```

2. Run a full-spectrum scan with all outputs

```
./flatscan -f target.bin --report-pack ./output_dir
```

3. Stream compact JSON to a pipeline

```
./flatscan -f target.bin --output-format jsonl | jq .
```

<https://github.com/Masriyan/FlatScan>